Veyo

One workspace for all your AI coding agents · one-page summary

WHAT IT IS

A Windows desktop workspace for running several AI coding agents (Claude Code, Codex and others) side by side. It has real terminals, a code editor and a usage dashboard. Agents run locally with the user's own logins.

PROBLEM

Developers now run multiple coding agents at once. They end up with scattered terminal windows, no view of what each agent is doing, no safe way to let two agents work on one repo, and no warning before they hit plan limits mid-task.

SOLUTION

- Launch and monitor many agents in one place.
- Forecast Claude and Codex usage limits, and pause or queue agents near a limit.
- Give each agent its own git worktree, then review all changes on one screen.
- Searchable history across all agents, with cost per project.

WHO IT'S FOR

Heavy agent users on Windows: indie developers, freelancers, dev agencies and small SaaS teams. Regulated industries (fintech, healthcare) come later, once audit and security features exist.

WHY IT CAN WIN

- **Neutral.** Anthropic and OpenAI won't build a view that works across their competitors.
- **Cheap to run.** Agents run on the user's machine, so there are no model costs.
- **Underserved platform.** Most agent tooling targets Mac first.
- **Gets harder to leave.** Cross-agent history, team playbooks and per-client cost data build up inside Veyo.

RISKS AND HOW WE ADDRESS THEM

Risk	Mitigation
Vendors add multi-agent panes	Compete on what they won't build: cross-vendor history, worktree isolation, team playbooks
Usage readings break when a CLI updates	Layered data sources, daily contract tests, a visible confidence label on every reading
Pre-launch stage (v0.1, no installer, accounts or pricing)	Private beta with about 30 users, weekly retention as the proof point

BUSINESS MODEL

Free for individuals. Paid Team plan for shared setups, team usage and cost dashboards, and permission policy, sold per seat. Start with agencies and small SaaS teams. Offer the first 50 teams a founding rate to test willingness to pay before building full billing.

ENGINEERING PRIORITIES

Reliable terminal startup, splitting `main.cjs` into modules, a local-versus-syncable state layer and a release pipeline. Then the differentiators: usage forecasting, worktree isolation and session history.

PLAN

1. **Weeks 1–4, make it shippable:** signed installer, auto-update, crash reporting, startup checks for missing folders and CLIs, landing page.
2. **Weeks 4–10, find users:** recruit 20–30 real users, interview ten, measure four-week retention.
3. **Weeks 10–16, launch:** Show HN and Product Hunt, led by a 30-second demo of three agents and a limit warning.
4. **Months 4–8, monetize:** accounts, billing and the Team plan. Sync only metadata, never code or CLI credentials.
5. **Month 8 onward:** Mac and Linux, audit and security features, partnerships.

> **Metric that matters most:** weekly active users still using it after four weeks.
> **Investor view today:** interesting, not fundable yet. The question is company or feature. A few paying teams with growing usage would change the answer.
> **Next step:** get ten real users on a signed build and measure how many come back.

Based on the product and internal analysis. It contains no market data, so verify the claims before sharing outside the team.